CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY (CHARUSAT)

Chandubhai S. Patel Institute of Technology (CSPIT)

Department of Computer Science & Engineering

ITUE203 – Web Development Frameworks

PRACTICAL 8

MySQL Schema Design, ER Model, PDO Connectivity and Prepared Statements

Student Name	Shreyas
Roll Number	25CS023
Student ID	25CS023
Email	25cs023@charusat.edu.in
Course	ITUE203 – Web Development Frameworks
Programme	B.Tech Computer Science & Engineering
Semester / Academic Year	3 / 2026-27
Date of Submission	
Faculty Signature	

1. Aim and Objectives

To design a normalised MySQL database for StudentHub with the tables students, events and registrations, import it with phpMyAdmin, and connect to it securely from PHP using PDO and prepared statements.

- Model the data with an ER diagram and normalise the tables to Third Normal Form (3NF).
- Define correct primary keys, foreign keys, unique keys, indexes, CHECK constraints and data types.
- Write an SQL dump with fictional seed data that imports in phpMyAdmin without manual steps.
- Create db.php: a PDO connection with exception mode, UTF-8, native prepared statements and error handling that never shows credentials or database errors in the browser.
- Create test_connection.php that proves the connection and reads all three tables with prepared statements.
- Advanced extension: add stored procedures; intermediate extension: PDO with prepared statements.
- Test every part and record only results that were actually executed.

2. Problem Definition

Official problem definition: *"Design the database for StudentHub with students, events and registrations. Create PHP database connection using MYSQL statements."*

StudentHub lets students register for college events. Until now the practicals stored data in static pages and JSON files. A real portal needs a database in which one student can register for many events and one event can have many students, without storing the same student or event details again and again. The database must refuse invalid data (for example a registration for a student who does not exist), and the PHP code must connect to it in a way that is safe against SQL injection and does not reveal the server's settings when something goes wrong.

Answers to the key questions of the practical:

Key question	Answer in this practical
Is the schema normalised properly?	Yes. All three tables are in 3NF: every column holds one value, every key is a single column, and no non-key column depends on another non-key column. The many-to-many relationship is resolved by registrations (Section 4.3).
Are primary keys and foreign keys correctly defined?	Each table has an AUTO_INCREMENT primary key; registrations.student_id and registrations.event_id are foreign keys with ON DELETE/ON UPDATE CASCADE. The tests confirmed that invalid references are rejected with MySQL error 1452 (Section 9).
Is the database connection handled securely?	Yes. PDO in exception mode with native prepared statements; no password in the repository; the real error is written to the PHP error log while the page shows only a general message (Sections 6–8).

3. Requirements and Tools

Item	Version / use
XAMPP	Windows package used for PHP, MariaDB, Apache and phpMyAdmin
PHP	8.2.12 (XAMPP) with the pdo_mysql extension; built-in server on 127.0.0.1:8608
Database server	MariaDB 10.4.32 (XAMPP's MySQL), port 3306
phpMyAdmin	5.2.1 – import of schema.sql and inspection of the tables
Browser	Microsoft Edge 156 – all screenshots
Testing	PHP CLI (tests/test_db.php) and Python 3.12 standard library (tests/test_page.py)
Operating system	Windows 11

Project folder: Practical-8 in the StudentHub repository. The practical uses its own database, studenthub_prac8, so the main StudentHub database and the earlier practical8_db are not changed.

4. Database Design

4.1 Tables

Table	Columns	Purpose
students	student_id (PK), student_code, full_name, email, branch, semester, created_at	One row per student
events	event_id (PK), title, event_date, venue, category, capacity, created_at	One row per event
registrations	registration_id (PK), student_id (FK), event_id (FK), registered_at, status	One row per student-event registration; resolves the many-to-many relationship

4.2 ER Diagram

A student makes zero or more registrations, and an event has zero or more registrations; each registration belongs to exactly one student and exactly one event. The diagram was drawn as SVG (docs/er-diagram.svg) and rendered to PNG with Microsoft Edge.

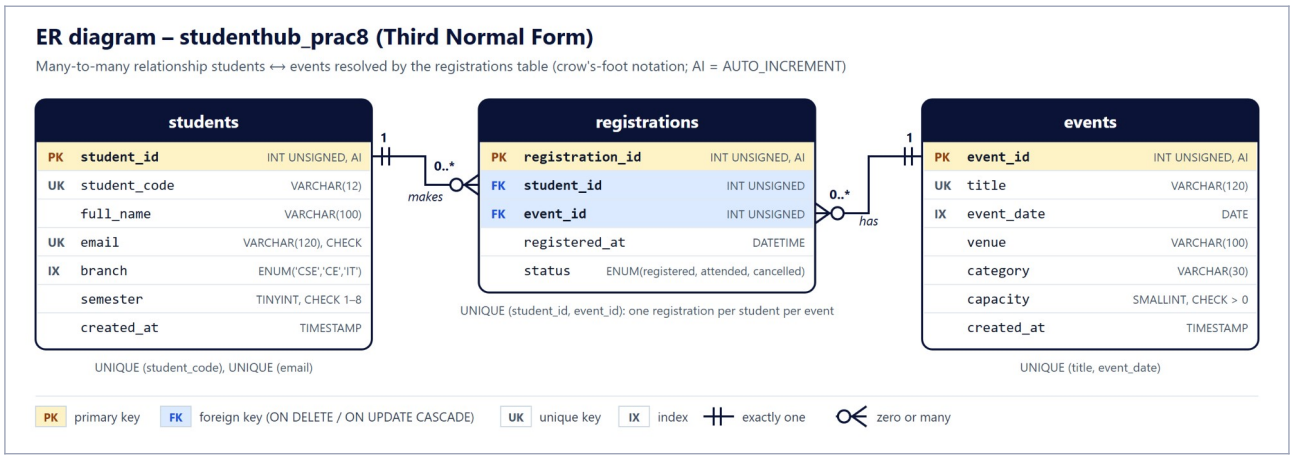


Figure 1: ER diagram of studenthub_prac8 (crow's-foot notation)

4.3 Normalisation to Third Normal Form

A first idea is often a single table such as the one below, where each row repeats the student and the event:

student_code	full_name	branch	event_title	event_date	venue	capacity	status
DEMO-S001	Aarohi Menon	CSE	Inter-college Hackathon	2026-08-22	Lab 1	4	registered
DEMO-S001	Aarohi Menon	CSE	Web Development Bootcamp	2026-08-30	Lab 3	30	attended
DEMO-S002	Eshan Kapoor	IT	Inter-college Hackathon	2026-08-22	Lab 1	4	registered

This design has update, insert and delete anomalies: changing a venue means changing every registration of that event, an event without registrations cannot be stored, and deleting the last registration of a student deletes the student. The schema was therefore normalised step by step:

Normal form	How the schema meets it
1NF	Every column holds one atomic value (no lists such as “events: 1, 3, 5”), there are no repeating groups, and every table has a primary key.
2NF	Every primary key is a single surrogate column (student_id, event_id, registration_id), so no column can depend on only part of a key.
3NF	No non-key column depends on another non-key column. Student details depend only on student_id and are stored only in students; event details depend only on event_id and are stored only in events. registrations stores just the two foreign keys and the facts about that registration (registered_at, status).

student_code and email are also unique, so they are candidate keys; making them UNIQUE keeps the design in 3NF (and in Boyce-Codd Normal Form) because every determinant is a key.

4.4 Keys, Constraints and Indexes

Element	Definition and reason
Primary keys	INT UNSIGNED AUTO_INCREMENT in every table – short, never changes, independent of names or codes.
Foreign keys	registrations.student_id → students.student_id and registrations.event_id → events.event_id, both ON DELETE CASCADE and ON UPDATE CASCADE: a registration cannot point to a missing student or event, and deleting a student also deletes that student's registrations.
Unique keys	students.student_code, students.email, events(title, event_date), registrations(student_id, event_id) – the last one stops a student registering twice for the same event.
CHECK constraints	semester BETWEEN 1 AND 8; email LIKE '%_@_%._%'; capacity > 0.
ENUM types	branch ('CSE', 'CE', 'IT') and status ('registered', 'attended', 'cancelled') allow only known values.
Indexes	students.branch (filter by branch), events.event_date (sorting by date), registrations.event_id (event rosters). MySQL indexes the foreign key student_id through the unique key (student_id, event_id).
Engine / character set	InnoDB (needed for foreign keys and transactions); utf8mb4 with utf8mb4_unicode_ci, so every name and symbol is stored correctly.

4.5 SQL Excerpts (schema.sql)

Database creation and safe re-import (child tables are dropped before parent tables):

```
15  SET NAMES utf8mb4;
16  SET time_zone = '+05:30';
17
18  CREATE DATABASE IF NOT EXISTS studenthub_prac8
19      CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
20  USE studenthub_prac8;
21
22  -- Drop in child-to-parent order so the foreign keys never block a re-import
23  DROP PROCEDURE IF EXISTS register_student_for_event;
24  DROP PROCEDURE IF EXISTS get_event_roster;
25  DROP TABLE IF EXISTS registrations;
26  DROP TABLE IF EXISTS events;
27  DROP TABLE IF EXISTS students;
```

The students table:

```
32  CREATE TABLE students (
33      student_id    INT UNSIGNED NOT NULL AUTO_INCREMENT,
34      student_code  VARCHAR(12)  NOT NULL,
35      full_name     VARCHAR(100) NOT NULL,
36      email         VARCHAR(120) NOT NULL,
37      branch        ENUM('CSE', 'CE', 'IT') NOT NULL,
38      semester      TINYINT UNSIGNED NOT NULL,
39      created_at    TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
40      PRIMARY KEY (student_id),
41      CONSTRAINT uq_students_code UNIQUE (student_code),
42      CONSTRAINT uq_students_email UNIQUE (email),
43      CONSTRAINT chk_students_semester CHECK (semester BETWEEN 1 AND 8),
44      CONSTRAINT chk_students_email CHECK (email LIKE '%_@_%._%'),
45      INDEX idx_students_branch (branch)
46  ) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE = utf8mb4_unicode_ci;
```

The registrations table with both foreign keys:

```
CREATE TABLE registrations (
    registration_id  INT UNSIGNED NOT NULL AUTO_INCREMENT,
    student_id       INT UNSIGNED NOT NULL,
    event_id         INT UNSIGNED NOT NULL,
    registered_at    DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    status           ENUM('registered', 'attended', 'cancelled') NOT NULL DEFAULT 'registered',
    PRIMARY KEY (registration_id),
    CONSTRAINT uq_registrations_student_event UNIQUE (student_id, event_id),
    INDEX idx_registrations_event (event_id),
    CONSTRAINT fk_registrations_student FOREIGN KEY (student_id)
        REFERENCES students (student_id) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_registrations_event FOREIGN KEY (event_id)
        REFERENCES events (event_id) ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE = utf8mb4_unicode_ci;
```

4.6 Fictional Seed Data

All sample data is invented: names, DEMO-S codes and example.com addresses do not belong to real students. It contains 8 students, 5 events and 12 registrations. The Inter-college Hackathon has a capacity of 4 and is already full, to demonstrate the capacity check; registration 7 is cancelled and registrations 5 and 6 are marked attended.

Event ID	Title	Date	Capacity	Registrations
1	Inter-college Hackathon	2026-08-22	4	4 (full)
2	Web Development Bootcamp	2026-08-30	30	3 (2 attended, 1 cancelled)
3	AI and Machine Learning Seminar	2026-09-15	100	3
4	Robotics Expo	2026-09-25	50	1
5	Annual Sports Week	2026-11-01	200	1

4.7 Stored Procedures (Advanced Extension)

register_student_for_event locks the event row, refuses an unknown or full event with SIGNAL SQLSTATE '45000', and otherwise inserts the registration and returns its ID. get_event_roster(event_id) returns the students registered for one event and is called from test_connection.php.

```
CREATE PROCEDURE register_student_for_event(IN p_student_id INT UNSIGNED, IN p_event_id INT UNSIGNED)
BEGIN
    DECLARE v_capacity SMALLINT UNSIGNED DEFAULT NULL;
    DECLARE v_taken INT DEFAULT 0;

    SELECT capacity INTO v_capacity FROM events WHERE event_id = p_event_id FOR UPDATE;
    IF v_capacity IS NULL THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Event not found';
    END IF;

    SELECT COUNT(*) INTO v_taken FROM registrations
    WHERE event_id = p_event_id AND status <> 'cancelled';
    IF v_taken >= v_capacity THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Event is full';
    END IF;

    INSERT INTO registrations (student_id, event_id) VALUES (p_student_id, p_event_id);
    SELECT LAST_INSERT_ID() AS registration_id;
END$$
```

5. Importing the Database with phpMyAdmin

- Start Apache and MySQL in the XAMPP Control Panel and open http://localhost/phpmyadmin.
- Open the Import tab, click Choose File and select Practical-8\schema.sql; keep the format SQL and click Import.
- phpMyAdmin reports “Import has been successfully finished, 17 queries executed.” The database studenthub_prac8 and its three tables appear in the left panel.

The import was carried out in Microsoft Edge: the file was chosen in phpMyAdmin's real file field and the Import button was pressed (automated through Edge's DevTools protocol so that it could be repeated). Figure 2 shows the database's Import tab with schema.sql selected and Figure 3 shows the result. The note #1007 “database exists” is expected on a re-import because of CREATE DATABASE IF NOT EXISTS.

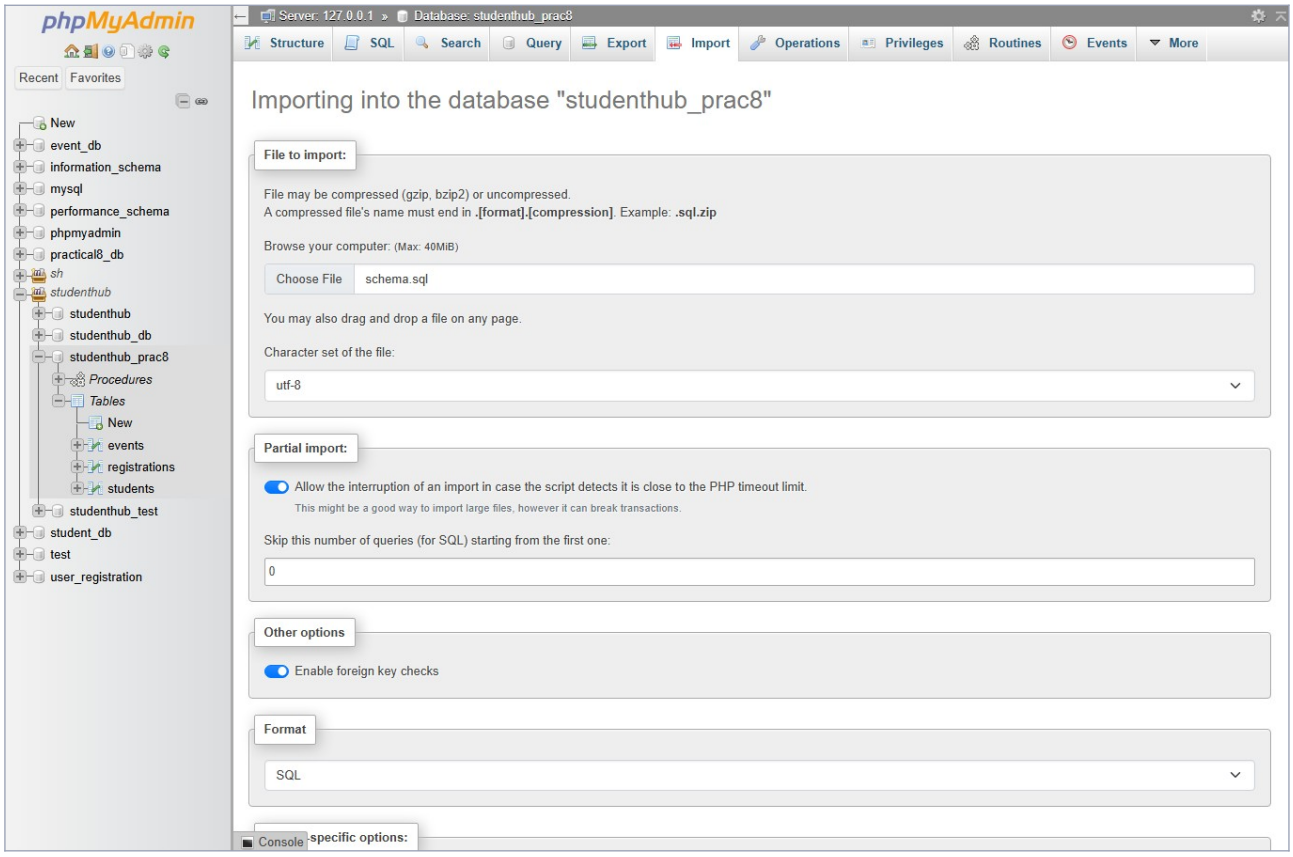


Figure 2: phpMyAdmin Import tab of studenthub_prac8 with schema.sql chosen

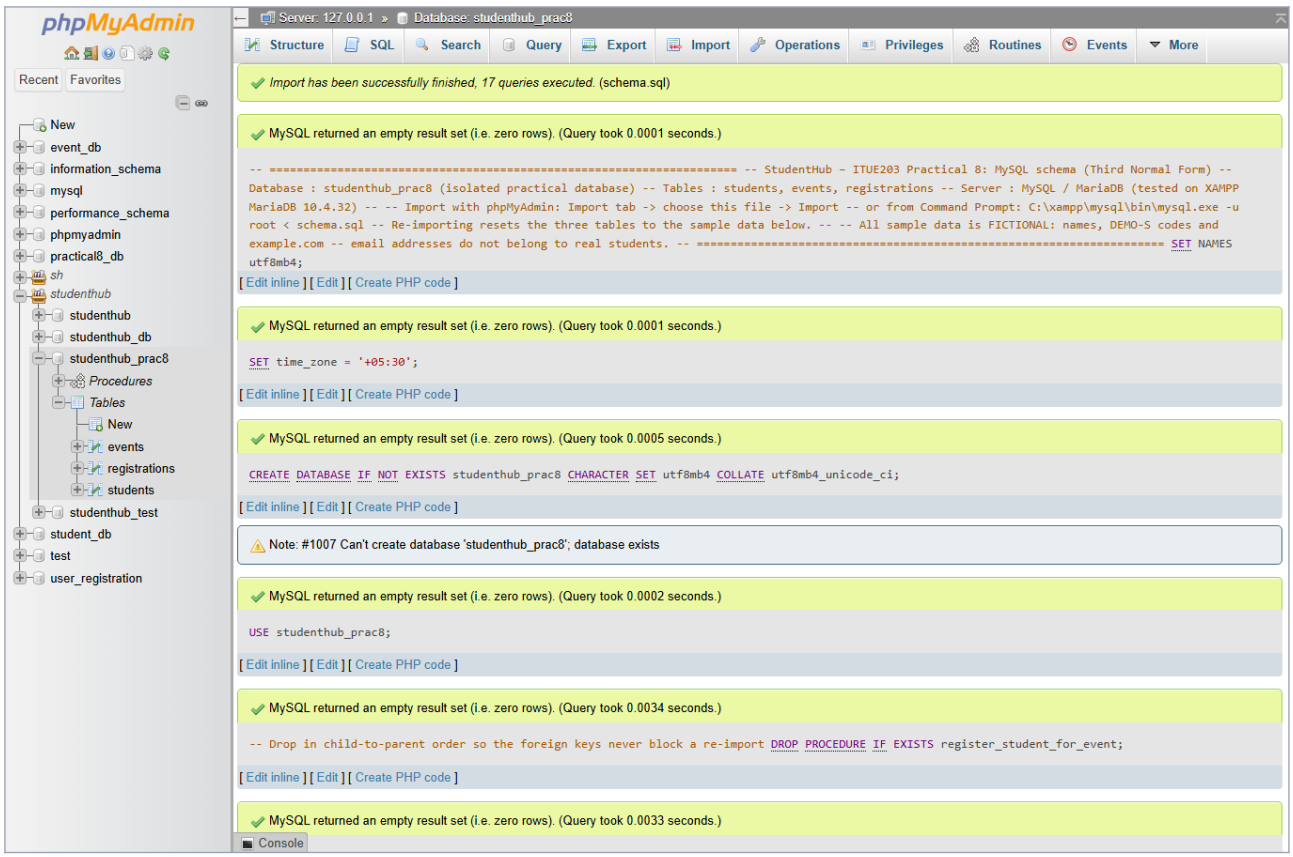


Figure 3: Import result – "Import has been successfully finished, 17 queries executed. (schema.sql)"

Observation: when the same file is imported from the server-level Import tab (no database selected), phpMyAdmin 5.2.1 additionally shows "Error: #1046 No database selected" after the SET, USE and procedure statements. Each statement itself succeeded, the database check afterwards found all tables, rows and procedures, and the same file imported with the MySQL command-line client (--show-warnings) or from the database's Import tab shows no such message. The messages therefore come from phpMyAdmin's own extra queries, not from schema.sql; this is explained in the README. The first import also showed warning #1280 because the primary keys had been given names that MySQL ignores; the names were removed and the warning no longer appears.

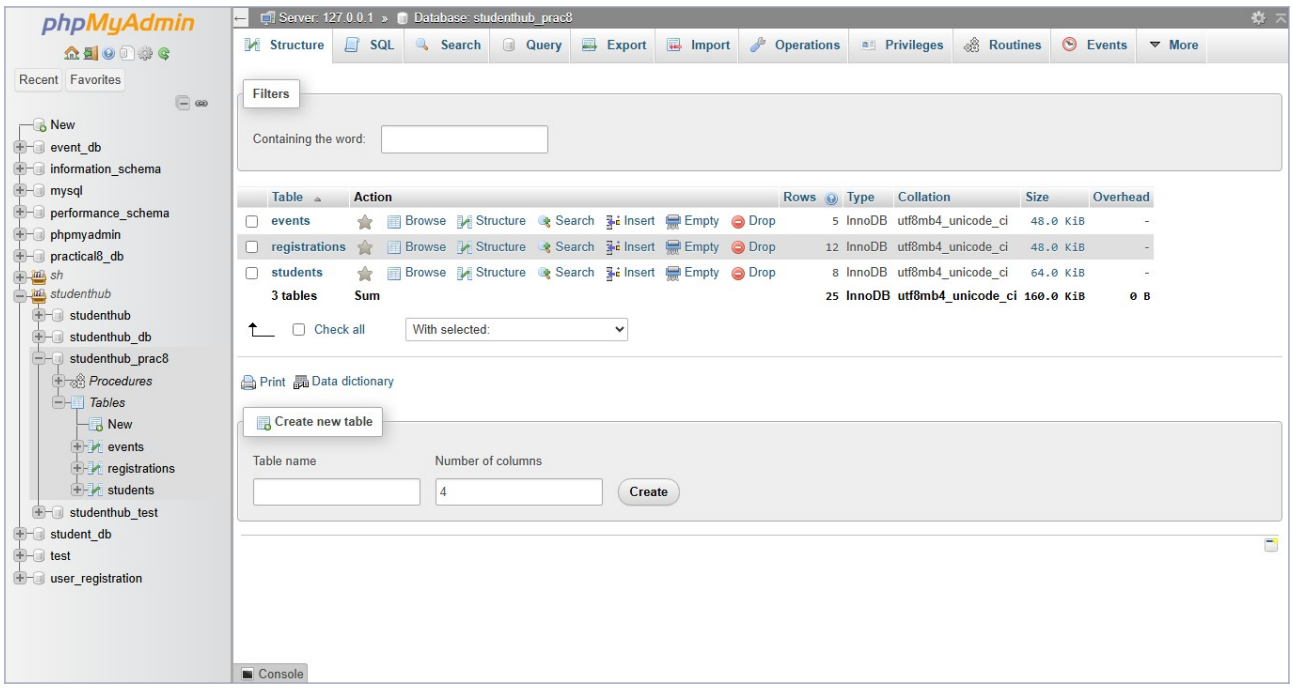


Figure 4: phpMyAdmin – the three InnoDB tables of studenthub_prac8 (events 5, registrations 12, students 8 rows)

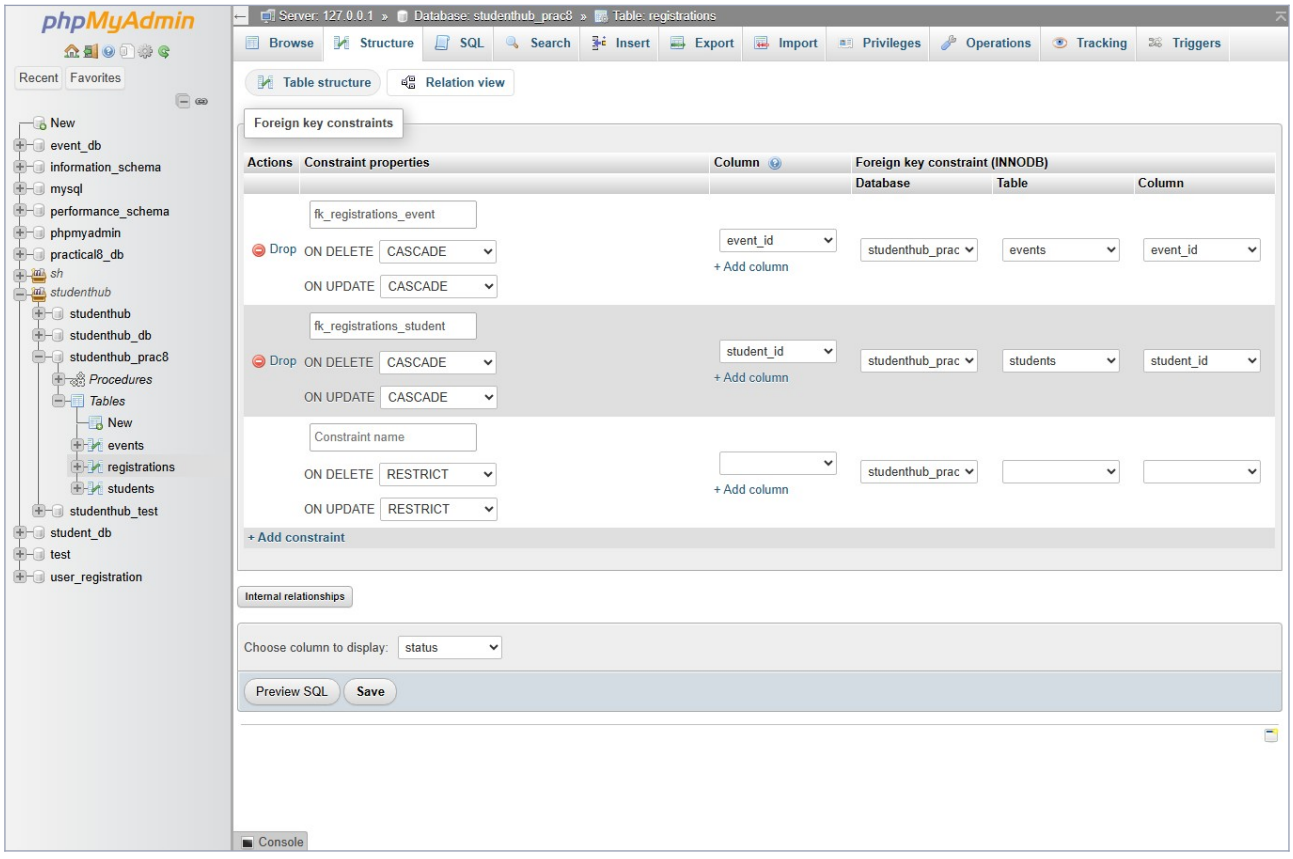


Figure 5: phpMyAdmin relation view of registrations – both foreign keys with ON DELETE / ON UPDATE CASCADE

6. Secure PDO Connection (db.php)

db.php contains only the connection. get_db() creates one PDO object, configures it and returns the same object to every caller:

```
28  // Returns one shared PDO connection. On failure the real error goes to the PHP error log and the
    caller
29  // receives a RuntimeException with a general message that contains no host, user or password.
30  function get_db(): PDO
31  {
32      static $pdo = null;
33      if ($pdo instanceof PDO) {
34          return $pdo;
35      }
36      $config = db_config();
37      $dsn = sprintf('mysql:host=%s;port=%d;dbname=%s;charset=utf8mb4', $config['host'], (int)
    $config['port'], $config['name']);
38      try {
39          $pdo = new PDO($dsn, (string) $config['user'], (string) $config['pass'], [
40              PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,   // every SQL error throws a
    PDOException
41              PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
42              PDO::ATTR_EMULATE_PREPARES   => false,                    // real server-side prepared
    statements
43              PDO::ATTR_STRINGIFY_FETCHES  => false,
44              // XAMPP's my.ini is not strict, so invalid values (e.g. branch 'XYZ') would be silently
    changed;
45              // strict mode for this connection makes MySQL reject them with an error instead
46              PDO::MYSQL_ATTR_INIT_COMMAND => "SET SESSION sql_mode =
    'STRICT_ALL_TABLES,ERROR_FOR_DIVISION_BY_ZERO,NO_ZERO_IN_DATE,NO_ZERO_DATE,NO_ENGINE_SUBSTITUTION'",
47          ]);
48      } catch (PDOException $exception) {
49          error_log('Practical 8 database connection failed: ' . $exception->getMessage());
50          throw new RuntimeException('The database connection could not be established.');
51      }
52      return $pdo;
53  }
```

Requirement	How db.php meets it
Isolated database	dbname=studenthub_prac8 in the DSN; the main StudentHub database is never used.
try / catch	new PDO(...) runs inside try; a PDOException is caught, logged with error_log() and replaced by a RuntimeException with a general message.
Exception mode	PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION, so every failed query throws instead of failing silently.
UTF-8	charset=utf8mb4 in the DSN; the tables also use utf8mb4_unicode_ci.
Prepared statements	PDO::ATTR_EMULATE_PREPARES => false: statements are prepared by MySQL itself and the values are sent separately.
Strict SQL mode	XAMPP's my.ini is not strict, so MySQL would silently change an invalid ENUM value to ''. The connection sets strict mode with PDO::MYSQL_ATTR_INIT_COMMAND, so such values are rejected (found by the tests).
No secrets in the repository	Only XAMPP's default local settings (root, empty password) are in the code. A different password goes in config.local.php, which is ignored by Git; config.example.php is the template.
Errors never shown	display_errors is turned off; the browser sees only a general message. Opening db.php directly returns 403.

7. Prepared Statements

A prepared statement sends the SQL text and the values to MySQL separately. MySQL compiles the SQL with placeholders (:branch or ?) first, and the value is then used only as data. Even a value such as CSE' OR '1'='1 can therefore never change the meaning of the query – it is simply compared with the branch column and matches nothing. All queries in test_connection.php use prepare() and execute(); the LIMIT value is bound as an integer with bindValue(..., PDO::PARAM_INT).

```
57  // Records of the three tables (LIMIT bound as an integer parameter)
58  $limit = 50;
59  $stmt = $pdo->prepare('SELECT student_id, student_code, full_name, email, branch, semester FROM
    students ORDER BY student_id LIMIT :limit');
60  $stmt->bindValue(':limit', $limit, PDO::PARAM_INT);
61  $stmt->execute();
62  $students = $stmt->fetchAll();
```

```
80  // A prepared statement with a value from the address (?branch=CE); the value is sent separately from
    the SQL
81  $stmt = $pdo->prepare('SELECT student_code, full_name, semester FROM students WHERE branch = :branch
    ORDER BY full_name');
82  $stmt->execute([':branch' => $branch]);
83  $byBranch = $stmt->fetchAll();
84
85  // Stored procedure (advanced extension) called through a prepared statement
86  $stmt = $pdo->prepare('CALL get_event_roster(:event_id)');
87  $stmt->execute([':event_id' => 1]);
88  $roster = $stmt->fetchAll();
89  $stmt->closeCursor();
```

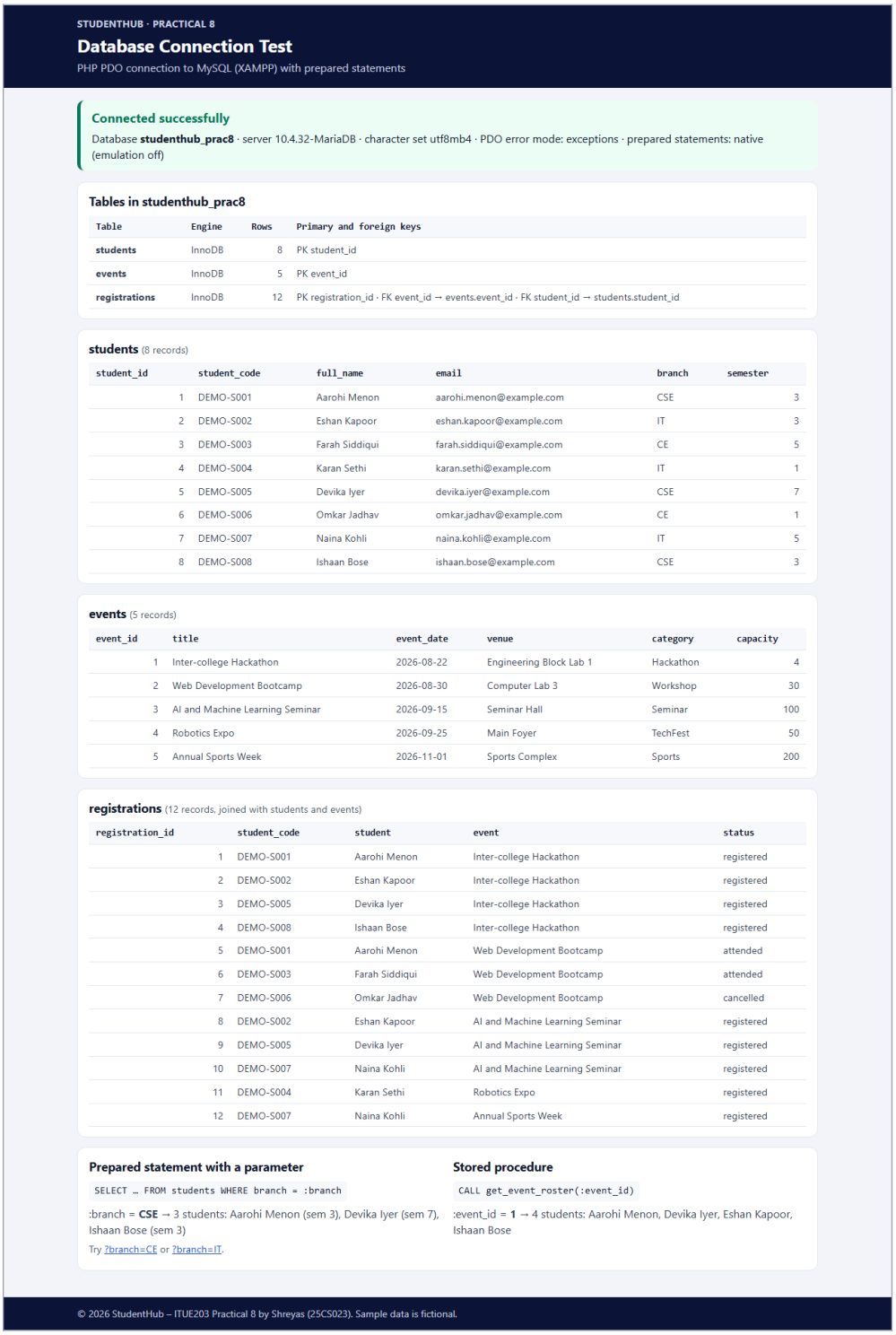
The branch value from the address (?branch=CE) is also checked against the list CSE, CE, IT before it is used, and table names (which cannot be bound as parameters) come only from a fixed list in the code. Every value printed in the page is escaped with htmlspecialchars().

```
90  } catch (PDOException $exception) {            // a query failed (PDOException must come first: it
    extends RuntimeException)
91      error_log('Practical 8 query failed: ' . $exception->getMessage());
92      $errorTitle = 'Database query failed';
93      $error = $exception->getCode() === '42S02'
94          ? 'Connected, but the tables were not found. Import schema.sql in phpMyAdmin and reload this
    page.'
95          : 'Connected, but a database query failed. The details were written to the PHP error log.';
96  } catch (RuntimeException $exception) {        // get_db() could not connect (already logged in
    db.php)
97      $errorTitle = 'Database connection failed';
98      $error = 'Could not connect to the studenthub_prac8 database. Check that MySQL is running in the
    XAMPP Control Panel and that schema.sql has been imported in phpMyAdmin.';
99  }
100 if ($error !== null) {
101     http_response_code(500);
102 }
```

PDOException is caught first because it is a subclass of RuntimeException; the tests found that the original order reported every query error as a connection error.

8. Output – Connection Test Page

test_connection.php was served by XAMPP's PHP (php -S 127.0.0.1:8608 router.php) and opened in Microsoft Edge. It shows the connection status, the server version and character set, the three table names with engine, row count and keys, all records of students, events and registrations (joined with names and titles), a prepared statement with a parameter and the stored-procedure roster.



STUDENTHUB · PRACTICAL 8

Database Connection Test

PHP PDO connection to MySQL (XAMPP) with prepared statements

Connected successfully

Database **studenthub_prac8** · server 10.4.32-MariaDB · character set utf8mb4 · PDO error mode: exceptions · prepared statements: native (emulation off)

Tables in studenthub_prac8

Table	Engine	Rows	Primary and foreign keys
students	InnoDB	8	PK student_id
events	InnoDB	5	PK event_id
registrations	InnoDB	12	PK registration_id · FK event_id → events.event_id · FK student_id → students.student_id

students (8 records)

student_id	student_code	full_name	email	branch	semester
1	DEMO-S001	Aarohi Menon	aarohi.menon@example.com	CSE	3
2	DEMO-S002	Eshan Kapoor	eshan.kapoor@example.com	IT	3
3	DEMO-S003	Farah Siddiqui	farah.siddiqui@example.com	CE	5
4	DEMO-S004	Karan Sethi	karan.sethi@example.com	IT	1
5	DEMO-S005	Devika Iyer	devika.iyer@example.com	CSE	7
6	DEMO-S006	Omkar Jadhav	omkar.jadhav@example.com	CE	1
7	DEMO-S007	Naina Kohli	naina.kohli@example.com	IT	5
8	DEMO-S008	Ishaan Bose	ishaan.bose@example.com	CSE	3

events (5 records)

event_id	title	event_date	venue	category	capacity
1	Inter-college Hackathon	2026-08-22	Engineering Block Lab 1	Hackathon	4
2	Web Development Bootcamp	2026-08-30	Computer Lab 3	Workshop	30
3	AI and Machine Learning Seminar	2026-09-15	Seminar Hall	Seminar	100
4	Robotics Expo	2026-09-25	Main Foyer	TechFest	50
5	Annual Sports Week	2026-11-01	Sports Complex	Sports	200

registrations (12 records, joined with students and events)

registration_id	student_code	student	event	status
1	DEMO-S001	Aarohi Menon	Inter-college Hackathon	registered
2	DEMO-S002	Eshan Kapoor	Inter-college Hackathon	registered
3	DEMO-S005	Devika Iyer	Inter-college Hackathon	registered
4	DEMO-S008	Ishaan Bose	Inter-college Hackathon	registered
5	DEMO-S001	Aarohi Menon	Web Development Bootcamp	attended
6	DEMO-S003	Farah Siddiqui	Web Development Bootcamp	attended
7	DEMO-S006	Omkar Jadhav	Web Development Bootcamp	cancelled
8	DEMO-S002	Eshan Kapoor	AI and Machine Learning Seminar	registered
9	DEMO-S005	Devika Iyer	AI and Machine Learning Seminar	registered
10	DEMO-S007	Naina Kohli	AI and Machine Learning Seminar	registered
11	DEMO-S004	Karan Sethi	Robotics Expo	registered
12	DEMO-S007	Naina Kohli	Annual Sports Week	registered

Prepared statement with a parameter

`SELECT … FROM students WHERE branch = :branch`

:branch = **CSE** → 3 students: Aarohi Menon (sem 3), Devika Iyer (sem 7), Ishaan Bose (sem 3)

Try ?branch=CE or ?branch=IT.

Stored procedure

`CALL get_event_roster(:event_id)`

:event_id = **1** → 4 students: Aarohi Menon, Devika Iyer, Eshan Kapoor, Ishaan Bose

© 2026 StudentHub – ITUE203 Practical 8 by Shreyas (25CS023). Sample data is fictional.

Figure 6: test_connection.php connected to studenthub_prac8 (screenshots/database-connection.png)

To test error handling, the same page was started with a wrong database password (environment variable P8_DB_PASS). The page answers with HTTP 500 and a general message only; the real error "SQLSTATE[HY000] [1045] Access denied for user …" was written to the PHP error log and does not appear in the page.

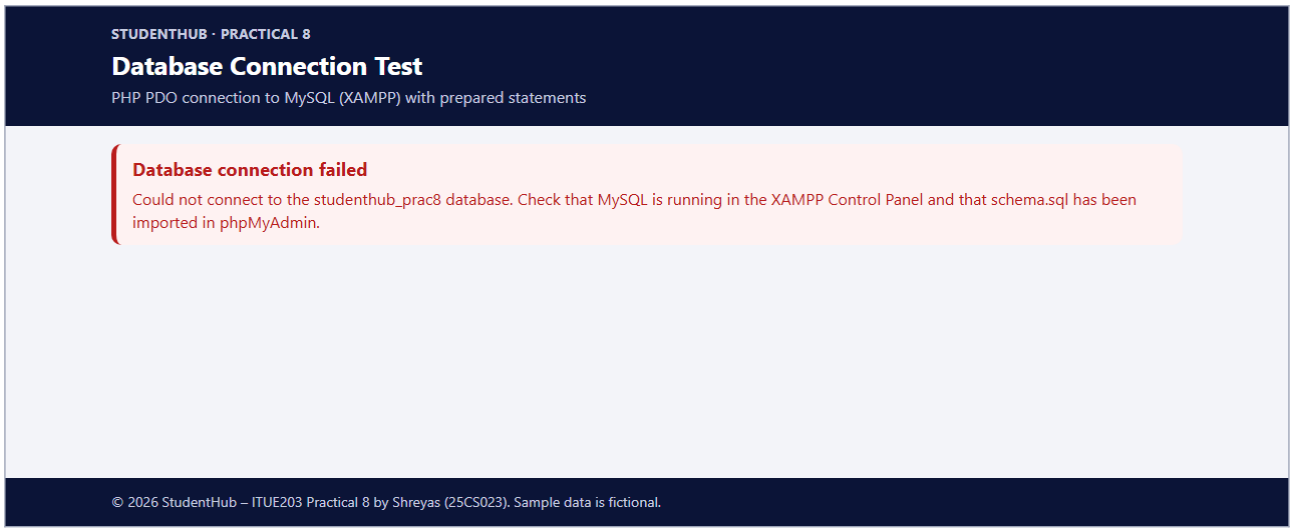


Figure 7: the same page with a wrong password – general message, no user name, host or SQLSTATE

9. Testing and Results

All tests below were executed on the final version (PHP 8.2.12, MariaDB 10.4.32, Windows 11). tests/test_db.php checks the database through db.php; tests/test_page.py runs php -l, test_db.php, the page tests on a PHP server and four error scenarios on separate servers. Every data change is made inside a transaction that is rolled back, and the database was re-imported with phpMyAdmin afterwards.

Test suite	Checks	Result
tests/test_db.php (PHP CLI)	44	44 passed, 0 failed
tests/test_page.py (Python, includes php -l and test_db.php)	44	44 passed, 0 failed

#	Area	Test	Actual result
1	SQL import	schema.sql imported through phpMyAdmin (server and database Import tab)	"Import has been successfully finished, 17 queries executed"
2	php -l	config.example.php, db.php, router.php, test_connection.php, tests/test_db.php	No syntax errors
3	Tables	exactly events, registrations, students; InnoDB; utf8mb4_unicode_ci; no NULL columns; data types	Pass
4	Keys	3 primary keys; 2 foreign keys with CASCADE; 4 unique keys; 3 indexes; 3 CHECK constraints	Pass
5	Seed data	8 / 5 / 12 rows; DEMO-S codes and example.com only; no orphan rows; no event over capacity	Pass

#	Area	Test	Actual result
6	PDO settings	studenthub_prac8; exception mode; emulation off; FETCH_ASSOC; utf8mb4; strict mode	Pass
7	Prepared statements	branch CSE/IT/CE → 3/3/2; injection text → 0 rows; positional ? placeholders; PARAM_INT; O'Neil stored correctly	Pass
8	Foreign keys	registration for student 999 or event 999	Rejected, error 1452
9	Unique keys	same student twice for one event; duplicate email	Rejected, error 1062
10	CHECK / ENUM	semester 9; email 'abc'; capacity 0; branch 'XYZ'	Rejected, errors 4025 / 1265
11	Cascade	delete student 1; change event_id 5 → 50	Registrations removed / updated
12	Procedures	roster of event 1; register into full event; unknown event; free place	4 rows; “Event is full”; “Event not found”; new ID
13	Page output	200; connection status; table names, row counts and keys; all records; JOIN; ?branch=IT and ?branch=CE	Pass
14	Invalid input	?branch=CSE' OR '1'='1 and ?branch[]=IT	Not accepted – falls back to CSE
15	Error handling	wrong database, wrong password, MySQL not reachable, tables missing	HTTP 500, general message, real error in the log
16	No leaks	page searched for root, SQLSTATE, Access denied, host, port, password	None found
17	Protected files	db.php, config.*.php, tests/, .gitignore, .htaccess, router.php, start-server.bat	403

Problems found by the tests and fixed

- **Invalid ENUM value accepted:** XAMPP's MariaDB runs without strict mode, so branch 'XYZ' was stored as an empty string. Fixed by setting strict SQL mode for every PDO connection in db.php.
- **Wrong error message:** PDOException extends RuntimeException, so a missing table was reported as “could not connect” and the query error was not logged. Fixed by catching PDOException first.
- **Import warning #1280:** MySQL ignores names given to primary keys and warned about them; the names were removed from schema.sql.
- **Test-script problems (not application bugs):** pdo_mysql reports the emulation setting as int(0) rather than false, and the Windows console could not print the → character; both were corrected in the test scripts.

Not tested: XAMPP Apache (the page was served by XAMPP's PHP built-in server), MySQL 8 (only MariaDB 10.4.32), and browsers other than Microsoft Edge.

10. Conclusion

The StudentHub database was designed with an ER model and normalised to Third Normal Form: students and events are stored once, and registrations links them with two foreign keys. Primary, foreign and unique keys, CHECK constraints and indexes make the database refuse invalid data, which the tests confirmed with the expected MySQL errors. schema.sql imports in phpMyAdmin in one step and contains only fictional data. db.php connects with PDO in exception mode, UTF-8 and native prepared statements, keeps the password out of the repository and shows only a general message when the connection fails. test_connection.php reads all three tables with prepared statements and stored procedures. Testing also showed why tests matter: they found a non-strict server setting and a wrong catch order that would otherwise have gone unnoticed.

11. References

- PHP Manual – PDO: https://www.php.net/manual/en/book.pdo.php
- PHP Manual – Prepared statements and stored procedures: https://www.php.net/manual/en/pdo.prepared-statements.php
- PHP Manual – PDO_MYSQL DSN: https://www.php.net/manual/en/ref.pdo-mysql.connection.php
- MariaDB Knowledge Base – CONSTRAINT (CHECK and FOREIGN KEY): https://mariadb.com/kb/en/constraint/
- MariaDB Knowledge Base – SQL_MODE: https://mariadb.com/kb/en/sql-mode/
- phpMyAdmin documentation – Import and export: https://docs.phpmyadmin.net/en/latest/import_export.html
- OWASP – SQL Injection Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- Project repository: https://github.com/TheEmperor08/StudentHub-ITUE203-25CS023 (folder Practical-8)